Case Study: COVID-19

COVID-19 Case Study:

Overview: *The COVID-19 pandemic, caused by the SARS-CoV-2 virus, emerged in late 2019 and rapidly spread globally, leading to significant health, economic, and social impacts. This unprecedented health crisis highlighted the crucial role of data analysis in managing such pandemics. By meticulously tracking and analyzing data on confirmed cases, recoveries, and deaths, policymakers and health professionals can make informed decisions to control the spread of the virus and allocate resources effectively.*

Dataset Details:
Dataset *(Open link in a new tab and download the dataset.)*
This case study utilizes three key datasets, each providing daily updates on different aspects of the pandemic for various countries and regions:

- **Confirmed Cases Dataset:** *Contains the cumulative number of confirmed COVID-19 cases per day for each country and region. The data spans from January 22, 2020, to May 29, 2021, with over 276 geographic entries.*
- **Deaths Dataset:** *Records the cumulative number of deaths attributed to COVID-19, structured similarly to the confirmed cases dataset. This provides crucial information for assessing the lethality and outbreak severity in different areas.*
- **Recovered Cases Dataset:** *Includes data on the cumulative number of individuals who have recovered from COVID-19, which is vital for understanding the disease's progression and the effectiveness of treatment protocols.*

Each dataset includes columns for Province/State, Country/Region, geographic coordinates (Lat, Long), and a series of dates representing daily cumulative totals.

Objective of the Case Study:
The primary objectives of this case study are:

- **Practical Application of Python:** *To demonstrate and enhance practical skills in using Python for data analysis, focusing on data manipulation, cleaning, and visualization.*
- **Insightful Data Analysis:** *To provide analytical insights into the dynamics of COVID-19's spread, recovery, and mortality rates across different regions and over time.*
- **Skill Development:** *To equip students with the knowledge to handle real-world data using Python libraries such as Pandas for data manipulation, Matplotlib for data visualization, and Numpy for numerical data processing.*

Analysis Questions:

1. ***Data Loading:*** *How do you load the COVID-19 datasets for confirmed cases, deaths, and recoveries into Python using Pandas?*

2. ***Data Exploration***

   1. *After loading the datasets, what is the structure of each dataset in terms of rows, columns, and data types?* `.info()`
   2. *Generate plots of confirmed cases over time for top countries.*
   3. *Generate plots of confirmed cases over time for China.*

3. ***Handling Missing Data:*** *Identify these missing values and replace them using a suitable imputation method such as forward filling for time-series data.*

4. ***Data Cleaning and Preparation:*** *Replace blank values in province column with "All Provinces"*

5. ***Independent Dataset Analysis***

   1. *Analyze the peak number of daily new cases in Germany, France, and Italy. Which country experienced the highest single-day surge, and when did it occur?*
   2. *Compare the recovery rates (recoveries/confirmed cases) between Canada and Australia as of December 31, 2020. Which country showed better management of the pandemic according to this metric?*
   3. *What is the distribution of death rates (deaths/confirmed cases) among provinces in Canada? Identify the province with the highest and lowest death rate as of the latest data point.*

6. ***Data Transformation***

   1. *Transform the 'deaths' dataset from wide format (where each column represents a date) to long format where each row represents a single date and columns are now country names, ensuring that the date column is in datetime format. How would this transformation be executed?* `.melt()`
   2. *What is the total number of deaths reported per country up to the current date?*
   3. *What are the top 5 countries with the highest average daily deaths?*
   4. *How have the total deaths evolved over time in the United States?*

7. *Data Merging*

1. *How would you merge the transformed datasets of confirmed cases, deaths, and recoveries on the 'Country/Region' and 'Date' columns to create a comprehensive view of the pandemic's impact?*
2. *Analyze the monthly sum of confirmed cases, deaths, and recoveries for countries to understand the progression of the pandemic.[From the merged dataset]*
3. *Redo the analysis in last question for United States, Italy and Brazil.*

8. *Combined Data Analysis*

1. *For the combined dataset, identify the three countries with the highest average death rates (deaths/confirmed cases) throughout 2020. What might this indicate about the pandemic's impact in these countries?*
2. *Using the merged dataset, compare the total number of recoveries to the total number of deaths in South Africa. What can this tell us about the outcomes of COVID-19 cases in the country?*
3. *Analyze the ratio of recoveries to confirmed cases for the United States on a monthly basis from March 2020 to May 2021. Which month experienced the highest recovery ratio, and what could be the potential reasons?*

```
>>> df = pd.DataFrame(
...     {
...         "a": [1, 2, 3, 4, 5, 6],
...         "b": [1, 1, 2, 3, 5, 8],
...         "c": [1, 4, 9, 16, 25, 36],
...     }
... )
>>> df
   a  b   c
0  1  1   1
1  2  1   4
2  3  2   9
3  4  3  16
4  5  5  25
5  6  8  36
>>> df.diff()
     a    b     c
0  NaN  NaN   NaN
1  1.0  0.0   3.0
2  1.0  1.0   5.0
3  1.0  1.0   7.0
4  1.0  2.0   9.0
5  1.0  3.0  11.0
```

pandas.DataFrame.clip

DataFrame.clip(*lower=None, upper=None, *, axis=None, inplace=False, **kwargs)

Trim values at input threshold(s).

```
>>> data = {"col_0": [9, -3, 0, -1, 5], "col_1": [-2, -7, 6, 8, -5]}
>>> df = pd.DataFrame(data)
>>> df
   col_0  col_1
0      9     -2
1     -3     -7
2      0      6
3     -1      8
4      5     -5
```

Clips per column using lower and upper thresholds:

```
>>> df.clip(-4, 6)
   col_0  col_1
0      6     -2
1     -3     -4
2      0      6
3     -1      6
4      5     -4
```

```
# Merging w.r.t Hypothetical Situations where Date is not Same for all Countries....
latest_totals = (
    merged.merge(latest_dates.rename('LatestDate'),
                 on = ['Country/Region'] , how = 'inner')
                 .query('Date == LatestDate')
)
latest_totals
```

	Country/Region	Date	Confirmed	Deaths	Recovered	LatestDate
493	Afghanistan	2021-05-29	70111	2899	57281	2021-05-29
987	Albania	2021-05-29	132297	2449	129215	2021-05-29
1481	Algeria	2021-05-29	128456	3460	89419	2021-05-29
1975	Andorra	2021-05-29	13693	127	13416	2021-05-29
2469	Angola	2021-05-29	34180	757	27646	2021-05-29
...	...	...	...	...	...	...
93365	Vietnam	2021-05-29	6908	47	2896	2021-05-29
93859	West Bank and Gaza	2021-05-29	307838	3492	300524	2021-05-29
94353	Yemen	2021-05-29	6731	1319	3399	2021-05-29
94847	Zambia	2021-05-29	94751	1276	91594	2021-05-29
95341	Zimbabwe	2021-05-29	38933	1584	36578	2021-05-29

94847	Zambia	2021-05-29	94751	1276	91594	2021-05-29
95341	Zimbabwe	2021-05-29	38933	1594	36578	2021-05-29

193 rows × 6 columns

```
top_5_countries = top_5_countries.index
top_5_countries
```

```
Index(['US', 'India', 'Brazil', 'France', 'Turkey'], dtype='object', name='Country/Region')
```

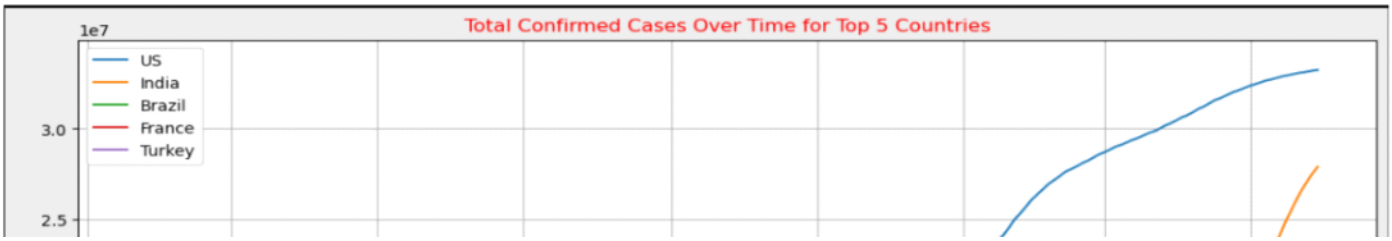
```
top_5_countries = top_5_countries.to_list()
top_5_countries
```

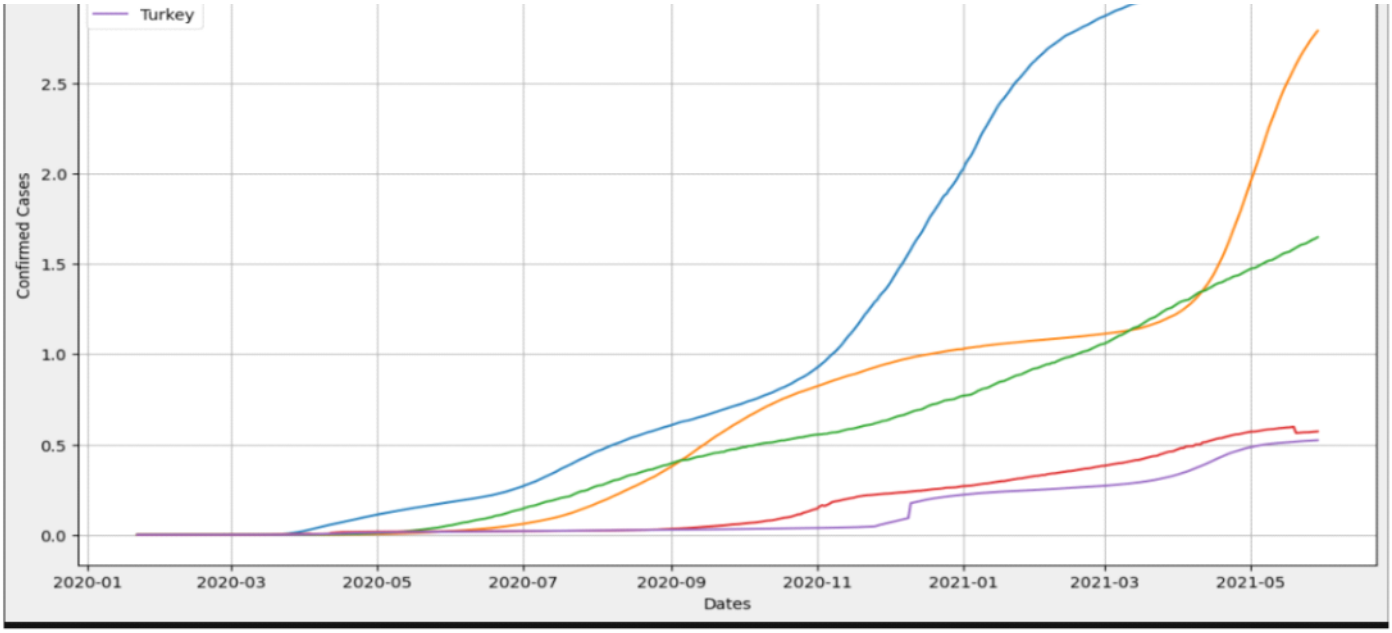
```
['US', 'India', 'Brazil', 'France', 'Turkey']
```

```
# Visualization
plt.figure(figsize = (14,8) , facecolor = 'grey')
for country in top_5_countries:
    top_countries = (merged[merged['Country/Region'] == country]).sort_values('Date')

top_countries
```

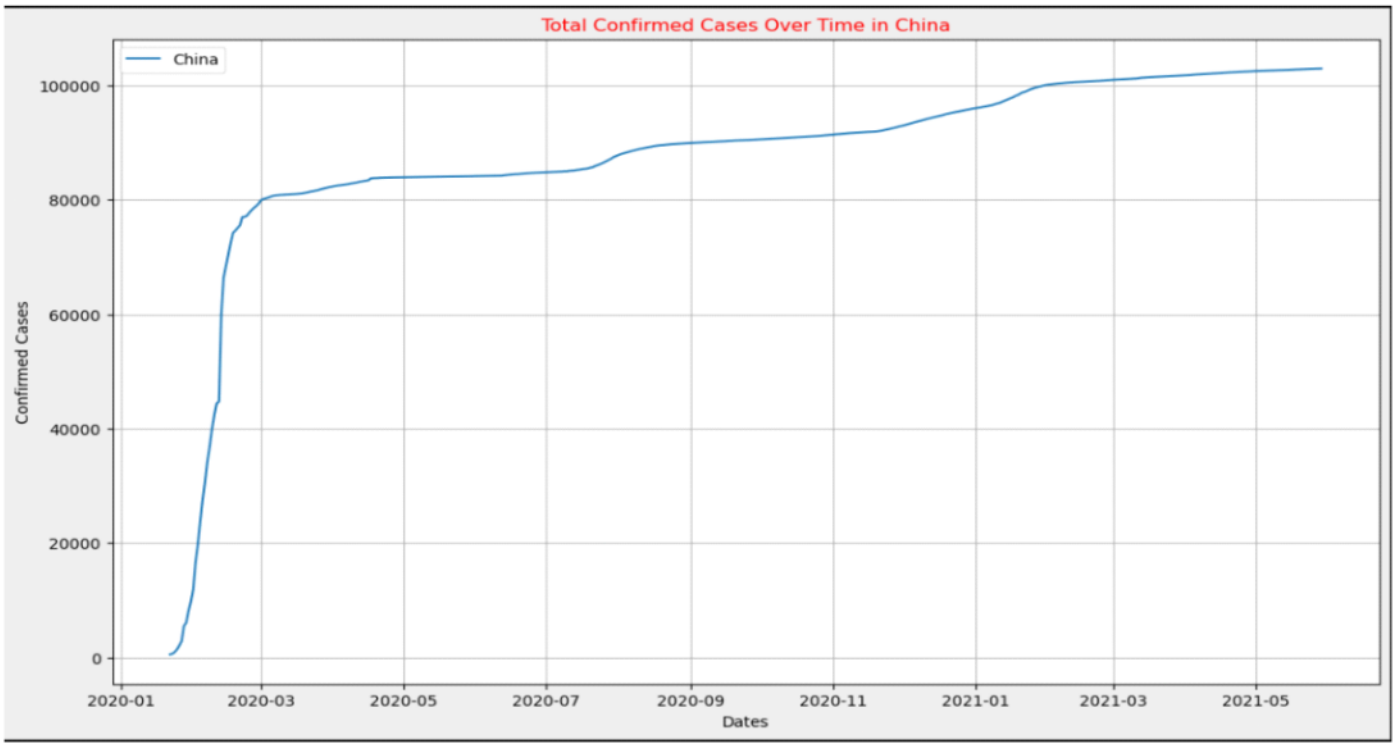
	Country/Region	Date	Confirmed	Deaths	Recovered
87932	Turkey	2020-01-22	0	0	0
87933	Turkey	2020-01-23	0	0	0
87934	Turkey	2020-01-24	0	0	0
87935	Turkey	2020-01-25	0	0	0
87936	Turkey	2020-01-26	0	0	0
...	...	...	...	...	...
88421	Turkey	2021-05-25	5203385	46621	5045508

```
# Line Plot Visualization
plt.figure(figsize = (14,8) , facecolor = '#E8E8E8')
for country in top_5_countries:
    top_countries = (merged[merged['Country/Region'] == country]).sort_values('Date')
    plt.plot(top_countries['Date'] , top_countries['Confirmed'] , label = country)

plt.title("Total Confirmed Cases Over Time for Top 5 Countries" , color = 'red')
plt.xlabel('Dates')
plt.ylabel('Confirmed Cases')
plt.grid()
plt.legend()
plt.show()
```






```python
# Generate plots of confirmed cases over time for China
# Line Plot Visualization
plt.figure(figsize = (14,8) , facecolor = '#E8E8E8')

top_countries = (merged[merged['Country/Region'] == 'China']).sort_values('Date')
plt.plot(top_countries['Date'] , top_countries['Confirmed'] , label = 'China')

plt.title("Total Confirmed Cases Over Time in China" , color = 'red')
plt.xlabel('Dates')
plt.ylabel('Confirmed Cases')
plt.grid()
plt.legend()
plt.show()
```



```python
china = confirmed_long[confirmed_long['Country/Region'] == 'China'].copy()
china.head(10)
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed
58	Anhui	China	31.8257	117.2264	2020-01-22	1
59	Beijing	China	40.1824	116.4142	2020-01-22	14
60	Chongqing	China	30.0572	107.8740	2020-01-22	6
61	Fujian	China	26.0789	117.9874	2020-01-22	1
62	Gansu	China	35.7518	104.2861	2020-01-22	0
63	Guangdong	China	23.3417	113.4244	2020-01-22	26
64	Guangxi	China	23.8298	108.7881	2020-01-22	2
65	Guizhou	China	26.8154	106.8748	2020-01-22	1
66	Hainan	China	19.1959	109.7453	2020-01-22	4
67	Hebei	China	39.5490	116.1306	2020-01-22	1

```python
china = china.groupby(['Date','Province/State'])['Confirmed'].sum().reset_index()
china
```

	Date	Province/State	Confirmed
0	2020-01-22	Anhui	1
1	2020-01-22	Beijing	14
2	2020-01-22	Chongqing	6
3	2020-01-22	Fujian	1
4	2020-01-22	Gansu	0
...	...	...	...
16791	2021-05-29	Tibet	1
16792	2021-05-29	Unknown	0
16793	2021-05-29	Xinjiang	980
16794	2021-05-29	Yunnan	352
16795	2021-05-29	Zhejiang	1364

16796 rows × 3 columns

```
china_wide = china.pivot(index = 'Date', columns = 'Province/State', values='Confirmed').sort_index()
china_wide
```

Province/State	Anhui	Beijing	Chongqing	Fujian	Gansu	Guangdong	Guangxi	Guizhou	Hainan	Hebei	...	Shandong	Shan
Date													
2020-01-22	1	14	6	1	0	26	2	1	4	1	...	2	
2020-01-23	9	22	9	5	2	32	5	3	5	1	...	6	
2020-01-24	15	36	27	10	2	53	23	3	8	2	...	15	
2020-01-25	39	41	57	18	4	78	23	4	19	8	...	27	
2020-01-26	60	68	75	35	7	111	36	5	22	13	...	46	
...	...	...	...	...	...	...	...	...	...	...	...	...	
2021-05-25	1004	1058	598	616	194	2413	275	147	188	1317	...	883	2
2021-05-26	1004	1059	598	617	194	2427	275	147	188	1317	...	883	2
2021-05-27	1004	1059	598	617	194	2428	275	147	188	1317	...	883	2
2021-05-28	1004	1059	598	618	194	2431	275	147	188	1317	...	883	2
2021-05-29	1004	1059	598	620	194	2432	275	147	188	1317	...	883	2

494 rows × 34 columns

```
# Top 5 Provinces
top_prov = china_wide.iloc[-1].nlargest(5)
top_prov # Series
```

```
Province/State
Hubei           68159
Hong Kong       11837
Guangdong        2432
Shanghai         2093
Heilongjiang     1612
Name: 2021-05-29 00:00:00, dtype: int64
```

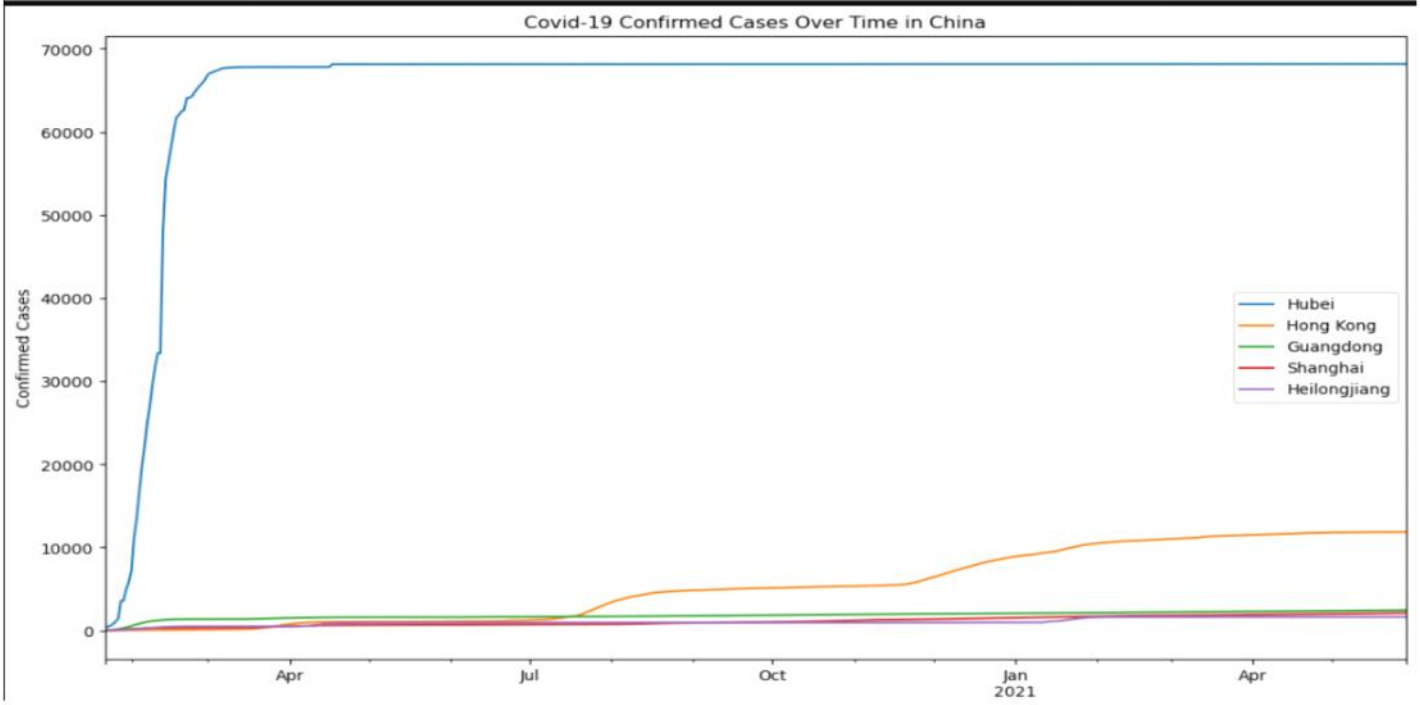
```
top_prov.index
```

```
Index(['Hubei', 'Hong Kong', 'Guangdong', 'Shanghai', 'Heilongjiang'], dtype='object', name='Province/State')
```

```
china_wide = china_wide[top_prov.index]
china_wide
```

Province/State	Hubei	Hong Kong	Guangdong	Shanghai	Heilongjiang
Date					
2020-01-22	444	0	26	9	0
2020-01-23	444	2	32	16	2
2020-01-24	549	2	53	20	4
2020-01-25	761	5	78	33	9
2020-01-26	1058	8	111	40	15
...	...	...	...	...	...
2021-05-25	68159	11835	2413	2078	1612
2021-05-26	68159	11836	2427	2079	1612

```python
ax = china_wide.plot(figsize = (14,8))
ax.set_title('Covid-19 Confirmed Cases Over Time in China')
ax.set_xlabel('Dates')
ax.set_ylabel('Confirmed Cases')
ax.legend(top_prov.index)
plt.show()
```




```python
# 5.1 Analyze the peak number of daily new cases in Germany, France, and Italy.
# Which country experienced the highest single-day surge, and when did it occur?

target_countries = ['Germany' , 'France' , 'Italy']
country_date_total = (
    merged.loc[merged['Country/Region'].isin(target_countries)]
    .groupby(['Country/Region','Date'])['Confirmed'].sum().reset_index()
    .sort_values(['Country/Region','Date'])
)
country_date_total
```

	Country/Region	Date	Confirmed
0	France	2020-01-22	0
1	France	2020-01-23	0
2	France	2020-01-24	2
3	France	2020-01-25	3
4	France	2020-01-26	3
...	...	...	...
1477	Italy	2021-05-25	4197892
1478	Italy	2021-05-26	4201827
1479	Italy	2021-05-27	4205970
1480	Italy	2021-05-28	4209707
1481	Italy	2021-05-29	4213055

1482 rows × 3 columns

```python
# Cumulative Confirmed Cases Convert -> Daily New Confirmed Cases [.diff]
country_date_total['DailyNew'] = (
    country_date_total.groupby('Country/Region')['Confirmed'].diff()
)
country_date_total
```

	Country/Region	Date	Confirmed	DailyNew
0	France	2020-01-22	0	NaN
1	France	2020-01-23	0	0.0
2	France	2020-01-24	2	2.0
3	France	2020-01-25	3	1.0
4	France	2020-01-26	3	0.0
...	...	...	...	...
1477	Italy	2021-05-25	4197892	3220.0
1478	Italy	2021-05-26	4201827	3935.0
1479	Italy	2021-05-27	4205970	4143.0
1480	Italy	2021-05-28	4209707	3737.0
1481	Italy	2021-05-29	4213055	3348.0

1482 rows × 4 columns

```python
# Identifying the missing Values
country_date_total[country_date_total['DailyNew'].isna()]
```

	Country/Region	Date	Confirmed	DailyNew
0	France	2020-01-22	0	NaN
494	Germany	2020-01-22	0	NaN
988	Italy	2020-01-22	0	NaN

```python
country_date_total['DailyNew'] = (
    country_date_total.groupby('Country/Region')['Confirmed'].diff()
    .fillna(country_date_total['Confirmed'])
)
country_date_total
```

	Country/Region	Date	Confirmed	DailyNew
0	France	2020-01-22	0	0.0
1	France	2020-01-23	0	0.0
2	France	2020-01-24	2	2.0
3	France	2020-01-25	3	1.0
4	France	2020-01-26	3	0.0
...	...	...	...	...
1477	Italy	2021-05-25	4197892	3220.0
1478	Italy	2021-05-26	4201827	3935.0
1479	Italy	2021-05-27	4205970	4143.0
1480	Italy	2021-05-28	4209707	3737.0

```python
# Identifying the missing Values
country_date_total[country_date_total['DailyNew'].isna()]
```

Country/Region	Date	Confirmed	DailyNew

```python
# Pivot Table
daily_new_matrix = (
    country_date_total.pivot(
        index = 'Date',
        columns = 'Country/Region',
        values = 'DailyNew'
    ).sort_index()
)
daily_new_matrix
```

Country/Region	France	Germany	Italy
Date			
2020-01-22	0.0	0.0	0.0
2020-01-23	0.0	0.0	0.0
2020-01-24	2.0	0.0	0.0
2020-01-25	1.0	0.0	0.0
2020-01-26	0.0	0.0	0.0
...	...	...	...
2021-05-25	3155.0	2578.0	3220.0
2021-05-26	12657.0	4473.0	3935.0
2021-05-27	13933.0	6949.0	4143.0

```python
daily_new_matrix['France'].nlargest(5)
```

```
Date
2021-04-11    117900.0
2020-11-02    106091.0
2021-04-08     97170.0
2020-11-07     86655.0
2021-04-04     80629.0
Name: France, dtype: float64
```

```python
daily_new_matrix['Germany'].nlargest(5)
```

```
Date
2020-12-30    49044.0
2021-01-07    45333.0
2020-12-22    36153.0
2020-12-15    33825.0
2020-12-23    33758.0
Name: Germany, dtype: float64
```

```python
daily_new_matrix['Italy'].nlargest(5)
```

```
Date
2020-11-13    40902.0
2020-11-07    39809.0
2020-11-12    37977.0
2020-11-06    37802.0
2020-11-14    37249.0
Name: Italy, dtype: float64
```

```python
daily_new_by_country = {}

# target_countries = ['Germany' , 'France' , 'Italy']

for country in target_countries:
    if country in daily_new_matrix.columns:
        s = daily_new_matrix[country]
        daily_new_by_country[country] = s

daily_new_by_country
```

```
{'Germany': Date
 2020-01-22       0.0
 2020-01-23       0.0
 2020-01-24       0.0
 2020-01-25       0.0
 2020-01-26       0.0
                 ...
 2021-05-25    2578.0
 2021-05-26    4473.0
 2021-05-27    6949.0
 2021-05-28    6169.0
 2021-05-29    4513.0
 Name: Germany, Length: 494, dtype: float64,
 'France': Date
 2020-01-22        0.0
 2020-01-23        0.0
 2020-01-24        2.0
 2020-01-25        1.0
 2020-01-26        0.0
                  ...
 2021-05-25     3155.0
 2021-05-26    12657.0
 2021-05-27    13933.0
```

```python
daily_new_matrix['Germany']
```

```
Date
2020-01-22       0.0
2020-01-23       0.0
2020-01-24       0.0
2020-01-25       0.0
2020-01-26       0.0
                ...
2021-05-25    2578.0
2021-05-26    4473.0
2021-05-27    6949.0
2021-05-28    6169.0
2021-05-29    4513.0
Name: Germany, Length: 494, dtype: float64
```

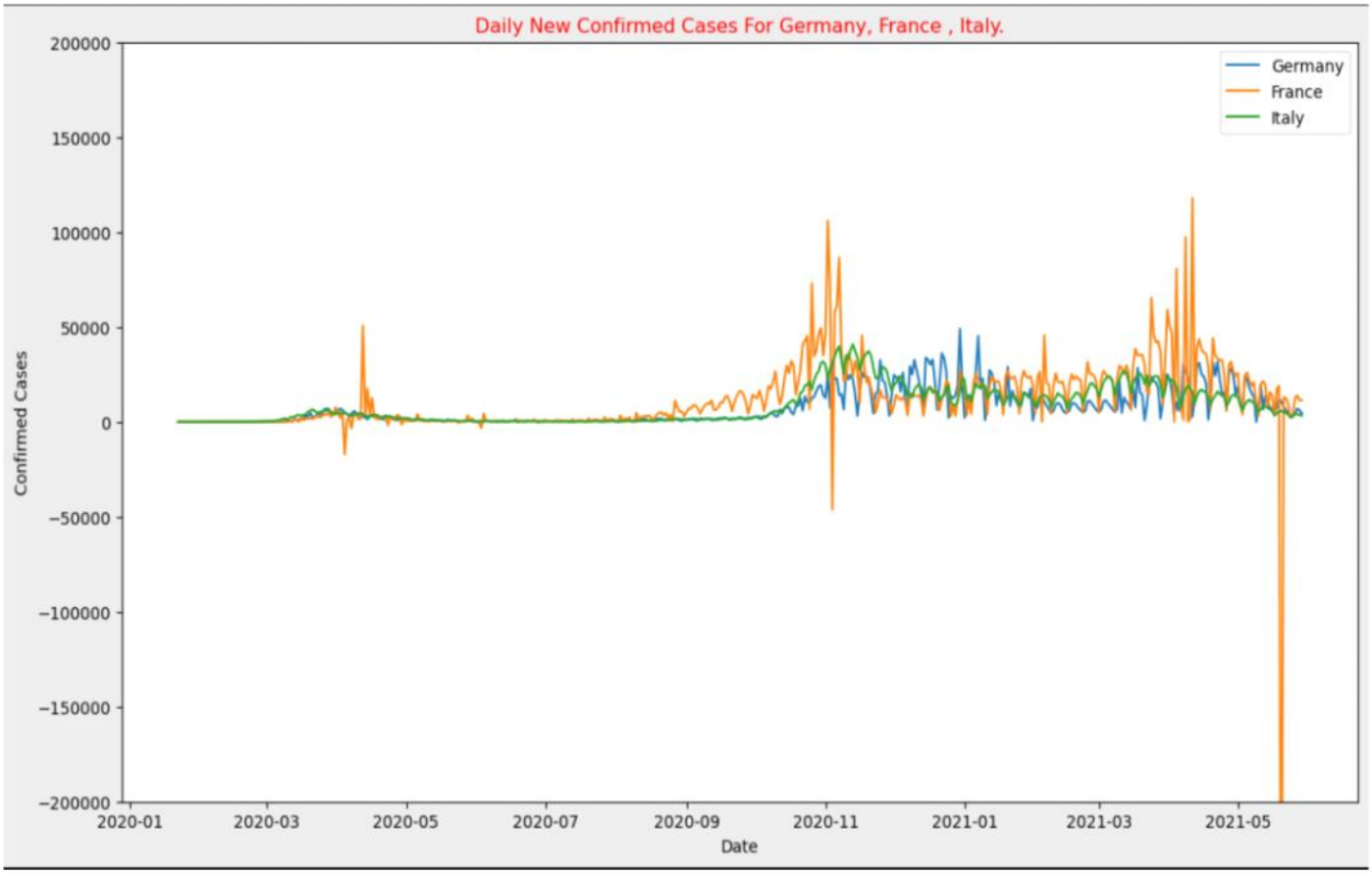
```python
daily_new_matrix['France']
```

```
Date
2020-01-22        0.0
2020-01-23        0.0
2020-01-24        2.0
2020-01-25        1.0
2020-01-26        0.0
                 ...
2021-05-25     3155.0
2021-05-26    12657.0
2021-05-27    13933.0
2021-05-28    11274.0
2021-05-29    11527.0
Name: France, Length: 494, dtype: float64
```

```python
# Peak {Country : (max_value , date} per date
peaks = {}
for country , series in daily_new_by_country.items():
    max_value = int(series.max())
    max_index = series.idxmax()
    peaks[country] = (max_value , max_index)

peaks
```

```
{'Germany': (49044, Timestamp('2020-12-30 00:00:00')),
 'France': (117900, Timestamp('2021-04-11 00:00:00')),
 'Italy': (40902, Timestamp('2020-11-13 00:00:00'))}
```

```python
# Visualizations
plt.figure(figsize = (14,8) , facecolor = '#E8E8E8')
for country , series in daily_new_by_country.items():
    plt.plot(series.index , series.values , label = country)

plt.title('Daily New Confirmed Cases For Germany, France , Italy.' , color = 'red')
plt.xlabel('Date')
plt.ylabel('Confirmed Cases')
plt.ylim(-200000 , 200000)
plt.legend()
plt.show()
```



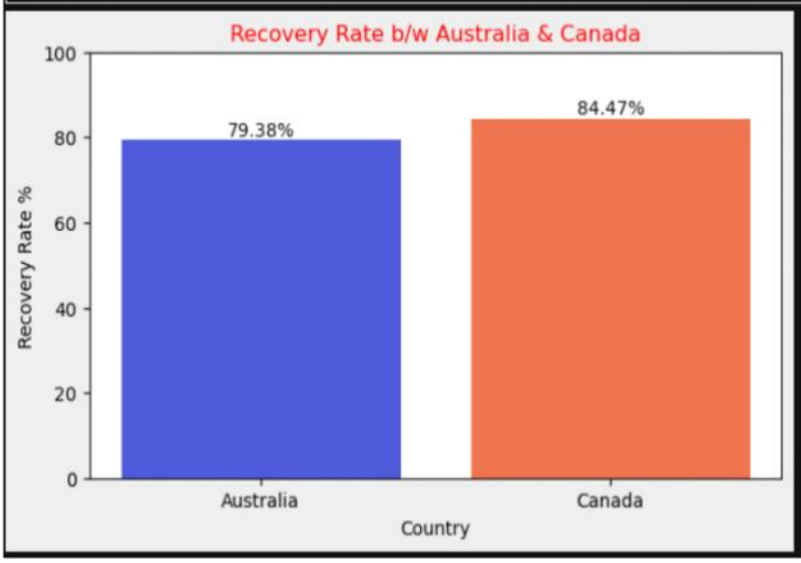
```python
# 5.2 Compare the recovery rates (recoveries/confirmed cases) between Canada and Australia
# as of December 31, 2020. Which country showed better management of the pandemic according to this metric?

countries = ['Canada' , 'Australia']
target_date = pd.Timestamp('2020-12-31')

# .loc[rows,cols]
recovery_rate = (
    merged.loc[(merged['Country/Region'].isin(countries)) & (merged['Date'] == (target_date)),
    ['Country/Region' , 'Recovered' , 'Confirmed']]
)
recovery_rate
```

	Country/Region	Recovered	Confirmed
4296	Australia	22565	28425
16152	Canada	493638	584409

```python
recovery_rate['RecoveryRate%'] = (recovery_rate['Recovered'] / recovery_rate['Confirmed'] * 100).round(2)
recovery_rate
```

	Country/Region	Recovered	Confirmed	RecoveryRate%
4296	Australia	22565	28425	79.38
16152	Canada	493638	584409	84.47

```python
# Visualization [Column Chart] [BarPlot]
plt.figure(figsize = (7,4) , facecolor = '#E8E8E8')
bars = plt.bar(recovery_rate['Country/Region'] , recovery_rate['RecoveryRate%'],
               color = ['#4D59D6' , '#E9724D'])
plt.title('Recovery Rate b/w Australia & Canada' , color = 'red')
plt.xlabel('Country')
plt.ylabel('Recovery Rate %')
plt.ylim(0,100)
plt.bar_label(bars , fmt = '%.2f%%')
plt.show()
```



```
# 5.3 What is the distribution of death rates (deaths/confirmed cases) among provinces in Canada?
# Identify the province with the highest and lowest death rate as of the latest data point.
confirmed_long[confirmed_long['Country/Region'] == 'Canada']
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed
39	Alberta	Canada	53.9333	-116.5765	2020-01-22	0
40	British Columbia	Canada	53.7267	-127.6476	2020-01-22	0
41	Diamond Princess	Canada	0.0000	0.0000	2020-01-22	0
42	Grand Princess	Canada	0.0000	0.0000	2020-01-22	0
43	Manitoba	Canada	53.7609	-98.8139	2020-01-22	0
...	...	...	...	...	...	...
136118	Prince Edward Island	Canada	46.5107	-63.4168	2021-05-29	200
136119	Quebec	Canada	52.9399	-73.5491	2021-05-29	369728
136120	Repatriated Travellers	Canada	0.0000	0.0000	2021-05-29	13
136121	Saskatchewan	Canada	52.9399	-106.4509	2021-05-29	46464
136122	Yukon	Canada	64.2823	-135.0000	2021-05-29	84

7904 rows × 6 columns

```
deaths_long[deaths_long['Country/Region'] == 'Canada']
```

	Province/State	Country/Region	Lat	Long	Date	Deaths
39	Alberta	Canada	53.9333	-116.5765	2020-01-22	0
40	British Columbia	Canada	53.7267	-127.6476	2020-01-22	0
41	Diamond Princess	Canada	0.0000	0.0000	2020-01-22	0
42	Grand Princess	Canada	0.0000	0.0000	2020-01-22	0
43	Manitoba	Canada	53.7609	-98.8139	2020-01-22	0
...	...	...	...	...	...	...
136118	Prince Edward Island	Canada	46.5107	-63.4168	2021-05-29	0
136119	Quebec	Canada	52.9399	-73.5491	2021-05-29	11125
136120	Repatriated Travellers	Canada	0.0000	0.0000	2021-05-29	0
136121	Saskatchewan	Canada	52.9399	-106.4509	2021-05-29	537
136122	Yukon	Canada	64.2823	-135.0000	2021-05-29	2

7904 rows × 6 columns

```
canada_confirmed_cases = (
    confirmed_long[confirmed_long['Country/Region'] == 'Canada']
    .sort_values('Date')
    .groupby(['Province/State']).tail(1)[['Province/State','Confirmed']]
    .set_index(['Province/State'])
)
canada_confirmed_cases
```

Province/State	Confirmed
Repatriated Travellers	13
Quebec	369728
Prince Edward Island	200
Ontario	537138
Nunavut	648
Nova Scotia	5530
Northwest Territories	127
Manitoba	50499
New Brunswick	2191
Grand Princess	13
Diamond Princess	0
British Columbia	143581
Alberta	226855
Saskatchewan	46464

```
canada_deaths_cases = (
    deaths_long[deaths_long['Country/Region'] == 'Canada']
    .sort_values('Date')
    .groupby(['Province/State']).tail(1)[['Province/State','Deaths']]
    .set_index(['Province/State'])
)
canada_deaths_cases
```

Province/State	Deaths
Repatriated Travellers	0
Quebec	11125
Prince Edward Island	0
Ontario	8698
Nunavut	4
Nova Scotia	84
Northwest Territories	0
Manitoba	1045
New Brunswick	43
Grand Princess	0
Diamond Princess	1
British Columbia	1692
Alberta	2214
Saskatchewan	537

```python
# .join Concatenate Columns [V-LookUp] [Related Function in DAX]
canada = canada_deaths_cases.join(canada_confirmed_cases , how = 'inner')
canada
```

Province/State	Deaths	Confirmed
Repatriated Travellers	0	13
Quebec	11125	369728
Prince Edward Island	0	200
Ontario	8698	537138
Nunavut	4	648
Nova Scotia	84	5530
Northwest Territories	0	127
Manitoba	1045	50499
New Brunswick	43	2191
Grand Princess	0	13
Diamond Princess	1	0
British Columbia	1692	143581
Alberta	2214	226855
Saskatchewan	537	46464
Newfoundland and Labrador	6	1302
Yukon	2	84

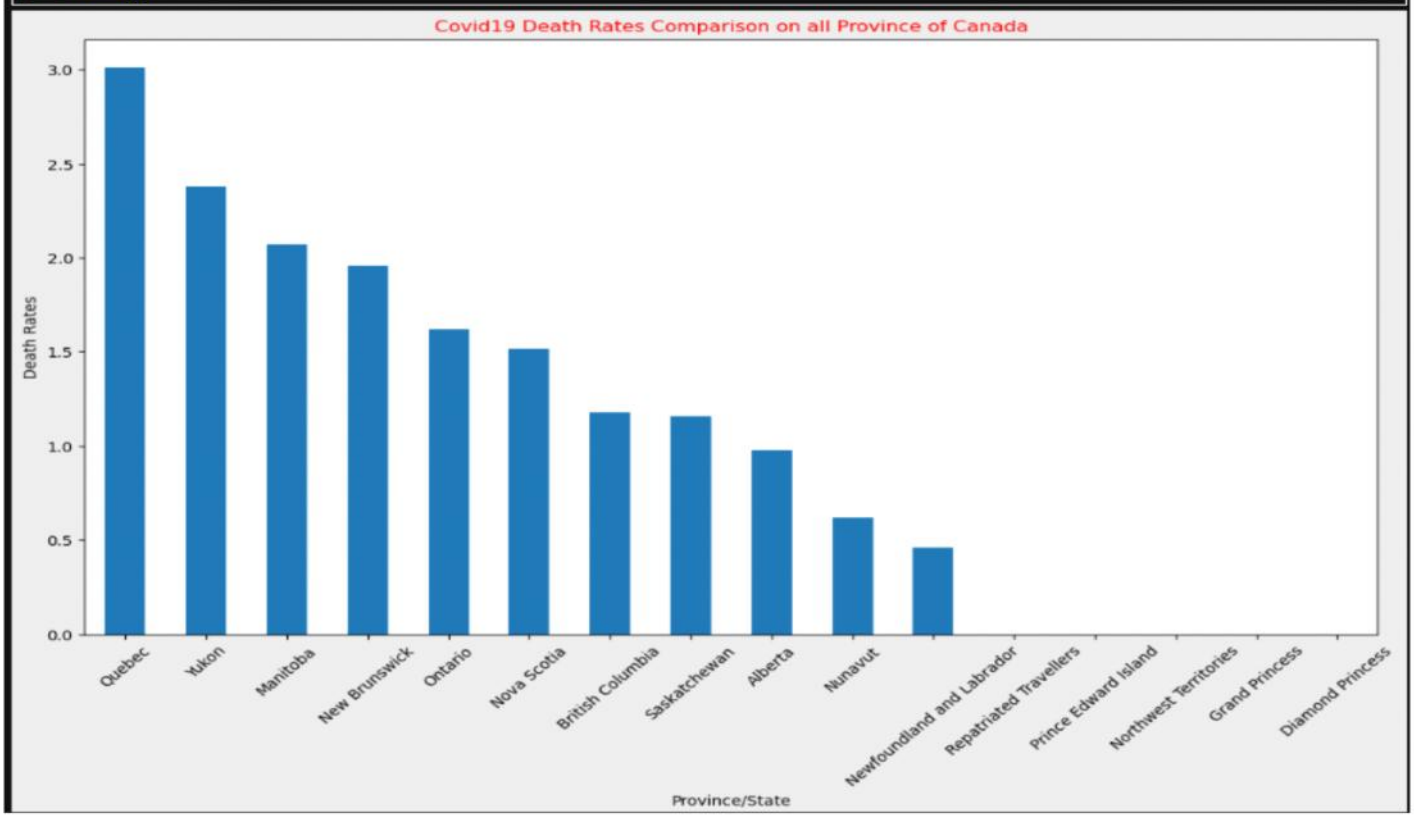
```python
# Death Rate [Deaths / Confirmed] %
canada['DeathRate'] = (canada['Deaths'] / canada['Confirmed'] * 100).round(2)
canada
```

Province/State	Deaths	Confirmed	DeathRate
Repatriated Travellers	0	13	0.00
Quebec	11125	369728	3.01
Prince Edward Island	0	200	0.00
Ontario	8698	537138	1.62
Nunavut	4	648	0.62
Nova Scotia	84	5530	1.52
Northwest Territories	0	127	0.00
Manitoba	1045	50499	2.07
New Brunswick	43	2191	1.96
Grand Princess	0	13	0.00
Diamond Princess	1	0	inf
British Columbia	1692	143581	1.18
Alberta	2214	226855	0.98
Saskatchewan	537	46464	1.16
Newfoundland and Labrador	6	1302	0.46
Yukon	2	84	2.38

```
# Death Rate [Deaths / Confirmed] %
import numpy as np
canada['DeathRate'] = (canada['Deaths'] / canada['Confirmed'] * 100).round(2).replace([np.inf],0)
canada
```

Province/State	Deaths	Confirmed	DeathRate
Repatriated Travellers	0	13	0.00
Quebec	11125	369728	3.01
Prince Edward Island	0	200	0.00
Ontario	8698	537138	1.62
Nunavut	4	648	0.62
Nova Scotia	84	5530	1.52
Northwest Territories	0	127	0.00
Manitoba	1045	50499	2.07
New Brunswick	43	2191	1.96
Grand Princess	0	13	0.00
Diamond Princess	1	0	0.00
British Columbia	1692	143581	1.18
Alberta	2214	226855	0.98
Saskatchewan	537	46464	1.16
Newfoundland and Labrador	6	1302	0.46
Yukon	2	84	2.38

```
# Visualization [Bar Plot]
plt.figure(figsize = (14,8) , facecolor = '#E8E8E8')
canada['DeathRate'].sort_values(ascending = False).plot(kind = 'bar') # High to Low
plt.title('Covid19 Death Rates Comparison on all Province of Canada' , color = 'red')
plt.xlabel('Province/State')
plt.ylabel('Death Rates')
plt.xticks(rotation=45)
plt.show()
```



```python
# 6.2 What is the total number of deaths reported per country up to the current date?
latest_dates = merged['Date'].max()
latest_dates
```

```
Timestamp('2021-05-29 00:00:00')
```

```python
latest_deaths = merged[merged['Date'] == latest_dates]
latest_deaths
```

	Country/Region	Date	Confirmed	Deaths	Recovered
493	Afghanistan	2021-05-29	70111	2899	57281
987	Albania	2021-05-29	132297	2449	129215
1481	Algeria	2021-05-29	128456	3460	89419
1975	Andorra	2021-05-29	13693	127	13416
2469	Angola	2021-05-29	34180	757	27646
...	...	...	...	...	...
93365	Vietnam	2021-05-29	6908	47	2896
93859	West Bank and Gaza	2021-05-29	307838	3492	300524
94353	Yemen	2021-05-29	6731	1319	3399
94847	Zambia	2021-05-29	94751	1276	91594
95341	Zimbabwe	2021-05-29	38933	1594	36578

193 rows × 5 columns

```python
# total number of deaths reported per country
total_deaths_by_country = (
    latest_deaths.groupby(['Country/Region'])['Deaths'].sum()
    .sort_values(ascending = False).reset_index() # Series -> DF
)
total_deaths_by_country.head(20)
```

	Country/Region	Deaths
0	US	594306
1	Brazil	461057
2	India	325972
3	Mexico	223455
4	United Kingdom	128037
5	Italy	126002
6	Russia	118781
7	France	109518
8	Germany	88413
9	Colombia	87747
10	Spain	79905
11	Iran	79741
12	Argentina	77108
13	Poland	73682
14	Peru	68978
15	South Africa	56363
16	Ukraine	52414

```python
# 6.3 What are the top 5 countries with the highest average daily deaths? ✗✗
average_daily_deaths_per_country  = (
    merged.groupby('Country/Region')['Deaths'].mean().round(2)
    .nlargest(5).reset_index()
)
average_daily_deaths_per_country
```

	Country/Region	Deaths
0	US	250234.91
1	Brazil	147013.33
2	India	89935.41
3	Mexico	87057.65
4	United Kingdom	58560.33

```python
# 6.3 What are the top 5 countries with the highest average daily deaths?
m = merged.sort_values(['Country/Region' , 'Date']).copy()
m.head()
```

	Country/Region	Date	Confirmed	Deaths	Recovered
0	Afghanistan	2020-01-22	0	0	0
1	Afghanistan	2020-01-23	0	0	0
2	Afghanistan	2020-01-24	0	0	0
3	Afghanistan	2020-01-25	0	0	0
4	Afghanistan	2020-01-26	0	0	0

```python
m['DailyDeaths'] = (
    m.groupby(['Country/Region'])['Deaths'].diff().fillna(0)
)
m
```

	Country/Region	Date	Confirmed	Deaths	Recovered	DailyDeaths
0	Afghanistan	2020-01-22	0	0	0	0.0
1	Afghanistan	2020-01-23	0	0	0	0.0
2	Afghanistan	2020-01-24	0	0	0	0.0
3	Afghanistan	2020-01-25	0	0	0	0.0
4	Afghanistan	2020-01-26	0	0	0	0.0
...	...	...	...	...	...	...

...	...	...	...	...	...	...
95337	Zimbabwe	2021-05-25	38706	1587	36517	1.0
95338	Zimbabwe	2021-05-26	38819	1589	36531	2.0
95339	Zimbabwe	2021-05-27	38854	1592	36541	3.0
95340	Zimbabwe	2021-05-28	38918	1592	36563	0.0
95341	Zimbabwe	2021-05-29	38933	1594	36578	2.0

95342 rows × 6 columns

```
average_daily_deaths_per_country = (
    m.groupby('Country/Region')['DailyDeaths'].mean().round(2)
    .nlargest(10).reset_index()
)
average_daily_deaths_per_country
```

	Country/Region	DailyDeaths
0	US	1203.05
1	Brazil	933.31
2	India	659.86
3	Mexico	452.34
4	United Kingdom	259.18
5	Italy	255.06
6	Russia	240.45
7	France	221.70
8	Germany	178.97
9	Colombia	177.63